EMIL

Evolutionary Market Intelligence Layer

HOW TO WORK ON EMIL

The operating manual for developing, extending and deploying the EMIL platform — cockpit, Command Center and EMIL Trade.

Version: Universe Round 15 · 4 September 2026

Prepared for the EMIL owner/operator. Keep alongside GAPS.md, README.md, STYLE_GUIDE.md and emil-trade/README.EMIL-TRADE.md in the repo.

1 · What EMIL is

EMIL is a professional financial intelligence, research and (authorization-gated) trading cockpit: a multi-agent trading brain with a Bloomberg-style research terminal around it, a separate Super-Admin Command Center, and — since 3 September 2026 — its own native trading platform, **EMIL Trade** (a rebranded clone of the Raptor terminal living inside this repo). The Universe loop: OBSERVE → VERIFY → UNDERSTAND → RESEARCH → SIMULATE → HEDGE → AUTHORIZE → EXECUTE → MONITOR → LEARN.

Cockpit (live)	https://serene-frangollo-a3c59c.netlify.app · Netlify site 8d9e2863-26ee-48c4-8e0f-a4a6f0448fb1 (serene-frangollo-a3c59c), team slug aby777333
Super admin	/command on the live site — admin role required, verified server-side per request
EMIL Trade (live)	https://emil-trade.netlify.app · Netlify site 56cac256-2896-473c-b43a-dd8105162792, git-connected with base directory emil-trade; Supabase leumpgkfillgeyyfptef (shared with Raptor: same logins/accounts)
GitHub	https://github.com/aby777333-pixel/EMIL.git — branch main; a push builds BOTH Netlify sites (EMIL Trade only when emil-trade/ changed)
Local folder	C:\Users\GIO4X\Documents\EMIL SEP 26 (cockpit at the root, terminal in emil-trade/)
Cockpit database	Supabase jnjxudsjyrftehwwdufs (Postgres + Prisma); app role emil_app via the transaction pooler (:6543, pgbouncer=true)
Key repo docs	GAPS.md = spec gap assessment + round log · STYLE_GUIDE.md = design system · README.md = setup · emil-trade/README.EMIL-TRADE.md = terminal provenance + deploy

2 · The golden rules (non-negotiable)

- **Do not break existing behaviour.** Every change is additive; verify the adjacent features after you touch anything.
- **Research data ≠ execution data.** Hub feeds are labelled (source, freshness, fetched-at, STALE, ETF PROXY) and never drive an order.
- **Research availability ≠ trading availability. Paper ≠ live ≠ demo.** Never label a future feature as live.
- **Free/open-first data, never scraping, never licence violations.** Twelve Data free tier = 8 credits/min: EMIL budgets 7, caches server-side, refuses locally with retry-after.
- **EMIL never tells anyone to buy or sell.** Briefs, reports, news scores and Ask EMIL are model assessments; the UI says so every time.
- **Kill switches are never permission-walled or flag-gated.** DISARM and EMERGENCY STOP stay available to every signed-in user; ARM, mode change and close-all are owner-only.
- **Secrets never reach the browser.** Provider/broker credentials are encrypted at rest (EMIL_SECRETS_KEY); API keys are stored as SHA-256 hashes and shown once.
- **Honest states everywhere:** add-a-key, coming soon, unavailable, cached, stale, proxy, paper.

3 · Architecture

Stack. Next.js 14 App Router (cockpit) · Prisma 6 on Supabase Postgres · NextAuth v4 credentials + JWT · Tailwind + shadcn/radix · TradingView Lightweight Charts v5 · Abacus chat-completions (gpt-5.4-mini, env ABACUSAI_API_KEY) for all LLM work · Netlify with @netlify/plugin-nextjs. EMIL Trade: Next.js 16 / React 19 / Tailwind 4 / Supabase (its own package.json, node_modules and netlify.toml inside emil-trade/).

Surfaces

- **Cockpit** (cyan): dashboard (+Morning Brief), /markets (+named watchlists, share links), /instruments (Instrument Master), /heatmap, /charts (compare, Bollinger, RSI pane, levels, layouts, research report), /correlation, /news (+AI impact scores), /alerts, /arm, /trades, /portfolio (consolidated exposure), /agents, /api-hub, /paper (protections shown), /backtest (+walk-forward, Monte Carlo), /options, /calendar, /journal, /risk (+circuit breakers), /capital, /strategies, /lab, /teach (Ask EMIL with live context), /trust, /settings. Sidebar deep links open EMIL Trade, Live TV and Live Chat in a new tab.

- **Command Center** (amber, admin-only): overview, customers·CRM, broker connections, data providers, API keys (+rotate), feature flags, demo environment, research ops, audit trail. Gated in app/command/layout.tsx by a per-request DB role check.
- **EMIL Trade** (emil-trade/): the terminal (/terminal), Live TV (■ chip + /terminal/tv), Live Chat (■ chip + /terminal/chat, Supabase realtime, 6 rooms), ABIN, scanner, hedge, EA builder, dealer desk… Every EMIL surface inside it opens the cockpit in a new tab (src/lib/emil-link.ts).
- **Platform API** /api/v1/[...path] — customer API keys issued and rotated from the Command Center.
- **Data layer** lib/data/hub.ts (fetch, cache, budgets, health) · lib/data/catalog.ts (providers) · lib/data/calendar.ts · lib/data/heat.ts · lib/instruments/* (master + normalisation) · research_cache table.
- **Intelligence layer** lib/brief.ts (Morning Brief) · lib/news-impact.ts · lib/report.ts (instrument reports) · lib/live-context.ts (Ask EMIL grounding) · lib/teach/llm.ts (llmJson).
- **Safety layer** lib/breakers.ts (circuit breakers, enforced on every /api/state read) · lib/execution/guards.ts (pre-trade protections) · lib/execution/router.ts (the only order path) · lib/flags.ts.

Key conventions

- Pages: app/<route>/page.tsx wraps app/<route>/_components/<name>-client.tsx in CockpitShell; export const dynamic = 'force-dynamic'.
- **Navigation trio:** a new page goes into components/cockpit/nav-items.ts AND command-palette.tsx (and components/command/shell.tsx for admin pages). External items carry external: true (new tab).
- **Any spelling resolves through the instrument master** (lib/instruments/catalog.ts): EURUSD, EUR/USD, gold, SPX, nifty… Research routes, watchlist adds and charts call toTwelveData()/resolveInstrument().
- New feature flags start OFF unless the feature is deliberately shipped ON (alerts_center, options_analytics, paper_trading_desk, morning_brief, news_impact_scoring, circuit_breakers, research_reports are ON; live_crypto_execution and autonomous_trading are OFF).

4 · Database practice

- Schema first: edit prisma/schema.prisma → npx prisma generate → apply the DDL on Supabase (Supabase MCP apply_migration / SQL editor as the postgres role) → code. **Never** prisma db push against the pooler (it hangs on pgbouncer).
- Tables created by the postgres role inherit emil_app grants through default privileges; when in doubt add explicit GRANT SELECT, INSERT, UPDATE, DELETE … TO emil_app.
- Migrations are additive. Use IF NOT EXISTS / ON CONFLICT DO NOTHING so a re-run is harmless. Recent examples: watchlists, chart_layouts, chart_levels, instrument_master, demo_environment, venue_orders guard columns.
- Feature flags live in feature_flags (id, key, label, description, enabled) — id is a required text; use ff_<key>.
- EMIL Trade's tables (chat_rooms, chat_messages) live in the Raptor Supabase project with RLS; the terminal repo itself was not changed.

5 · The standard change workflow

1. Inspect	Read GAPS.md and the code you will touch. Trace reuse. Understand before editing.
2. Extend	Smallest safe change. New feature = new files + minimal wiring, never rewrites of stable logic.
3. Schema first	schema.prisma → prisma generate → DDL on Supabase → code (section 4).
4. Type-check + build	npx tsc --noEmit -p tsconfig.json, then npx next build must exit clean. NEVER run next build while next dev is running (they share .next).
5. Verify live	Cockpit dev server: launch config emil-dev (port 3470). EMIL Trade dev: emil-trade-terminal (port 3461; needs emil-trade/.env.local with the Supabase vars). Sign in, exercise the feature and its neighbours, check the console.
6. Regression	Dashboard, /markets, /arm, /command load clean? Mobile not overflowing? Old features untouched?

7. Update GAPS.md	Move shipped items, add the round section, keep the roadmap honest.
8. Commit + push	Descriptive commit to main. CRLF warnings on Windows are normal.
9. Deploy cockpit	npx netlify api createSiteBuild --data '{"site_id":"8d9e2863-26ee-48c4-8e0f-a4a6f0448fb1","clear_cache":true}' — never deploy a locally built .next from Windows.
10. Deploy EMIL Trade	A push that touches emil-trade/ builds it automatically; force one with the same command and site_id 56cac256-2896-473c-b43a-dd8105162792. NEVER netlify deploy --build from the subfolder (the CLI takes the git root as project root and drops the server function → every page 404s).
11. Verify prod	Sign in on the live site (admin), spot-check the changed pages; same-origin fetches from the browser console are a fast way to prove APIs.

6 · Environment & secrets

DATABASE_URL	Supabase transaction pooler (:6543/postgres?pgbouncer=true&connection;_limit=1) as emil_app — session mode hits its client cap under Lambda.
NEXTAUTH_SECRET / NEXTAUTH_URL	Auth; set on Netlify.
ABACUSAI_API_KEY	All LLM work (Ask EMIL, briefs, reports, news scoring, journal reviews). Missing key = honest 503, never a fake answer.
EMIL_SECRETS_KEY	AES-256-GCM key for credentials at rest (enc:v1: prefix). Rotating it requires re-encrypting rows.
TELEGRAM_BOT_TOKEN / RESEND_API_KEY / EMAIL_FROM	Alert delivery (Telegram + email). Still unset by the owner → in-app only.
EMIL_PAPER_MAX_NOTIONAL_USD / EMIL_LIVE_MAX_NOTIONAL_USD	Per-order caps (defaults 25 000 / 1 000).
EMIL_MAX_QUOTE_AGE_MS, EMIL_MAX_QUOTE_LATENCY_MS, EMIL_MAX_SPREAD_BPS, EMIL_MAX_LIMIT_DEVIATION_PCT, EMIL_SLIPPAGE_ALERT_BPS, EMIL_MAX_ORDERS_PER_DAY	Execution protections (defaults 10 000 ms, 3 000 ms, 60 bps, 5 %, 25 bps, 200/day). Quote age refuses LIVE orders only — sandbox tickers stamp the last trade.
EMIL Trade (emil-trade/netlify.toml)	NEXT_PUBLIC_SUPABASE_URL/ANON_KEY (Raptor project), NEXT_PUBLIC_SITE_URL, NEXT_PUBLIC_EMIL_COCKPIT_URL, Twelve Data key. Local dev needs the same in emil-trade/.env.local (gitignored).

7 · Data Provider Hub rules

- Free/open-first, modular, honest. Providers are catalogued in lib/data/catalog.ts with licence notes, priority, fallback and health tests; all fetching goes through lib/data/hub.ts (timeout fetch, health stamps, research_cache, stale-serve labelled STALE).
- Twelve Data: 8 credits/min free. The board is 6 symbols (ETF proxies, labelled), watchlists cap at 8 DISTINCT symbols across all named lists, alerts ride watchlist quotes, charts/reports cost 1 credit per uncached series. A budget hit is a 429 with retry-after — and a WARN, not a trip, for the market-data breaker.
- Frankfurter (ECB fixings) powers FX rates and the FX heatmap (time series /v1/{from}..{to}?base=USD); CoinGecko powers crypto boards/heat; Forex Factory JSON powers the calendar (next-week feed 404s — handled); GDELT → Google News RSS for headlines.
- LLM cost is bounded by caching: ≤1 Morning Brief per user per 6 h, ≤1 news-scoring call per distinct headline batch per 30 min, ≤1 research report per user per symbol per 6 h, Ask EMIL live context cached 60 s.
- Adding a provider: catalog entry → bespoke health test in testProvider() → fetch through cachedFetch with an explicit TTL → label source + freshness in the UI.

8 · Auth, roles & safety systems

- Roles: trader | admin. requireAdmin() is a per-request DB check; the JWT role is a UI hint only. EmilState is ONE global row: ARM, mode change and close-all are admin-only; DISARM and EMERGENCY STOP are open to all.

- **Circuit breakers** (lib/breakers.ts): daily/weekly loss, drawdown from HWM, margin utilisation, open-position cap, consecutive losses, ±30 min high-impact news window, broker link, market-data health. Evaluated on every /api/state read (memo 45 s); a disarm-class trip sets armed=false, writes an emergency_events row (circuit_breaker), audits and notifies admins. Enforcement gated by flag circuit_breakers; positions are never touched.
- **Execution protections** (lib/execution/guards.ts) run inside the guarded router for every order: duplicate window, daily budget, breaker gate (live refused, paper noted), quote latency/age, spread cap for market orders, fat-finger limit deviation, realised slippage per fill with an audit alert.
- Live venue orders additionally need flag live_crypto_execution ON + ARMED + admin; paper venues (Deribit testnet, Gemini sandbox, Delta demo) are always allowed within the paper cap.
- 2FA (TOTP) and DB-backed rate limits on sign-in/sign-up exist. API keys rotate from Command → API Keys (old key revoked the moment the new one is issued).
- Demo environment (Command → Demo Environment): a dedicated demo TRADER login — rotate its password, reset its portfolio to the baseline; every action audited.

9 · Gotchas that cost hours

- The dev server talks to the PRODUCTION database. Test artefacts are real — clean them up.
- next build while next dev runs corrupts .next. Scripted import edits can produce ',,' or duplicate imports — always tsc --noEmit after them.
- EMIL Trade inside the repo: two lockfiles → Next infers the PARENT as root unless turbopack.root/outputFileTracingRoot are pinned (they are, in emil-trade/next.config.ts). Stale .next/dev after a root change → rm -rf .next.
- netlify deploy --build from emil-trade/ silently drops the Next server function (pages 404, static + edge middleware work). Use the git-connected build.
- Supabase Realtime: the sender's own INSERT can land before the channel is SUBSCRIBED — echo the inserted row optimistically (Live Chat does).
- Gemini sandbox: crossed/stale book, ticker ts = last trade, order ids > MAX_SAFE_INTEGER. Delta demo keys need EMIL's egress IP whitelisted (Netlify IPs vary).
- Netlify functions time out ~26 s: keep LLM batches small (news scoring caps at 24 headlines; brief ~18 s).
- Windows CRLF warnings on commit are normal; do not 'fix' them repo-wide.

10 · Where EMIL stands (Round 15, 2026-09-04)

Shipped and live

- Rounds 1–5: hub, markets, charts, correlation, news, alerts, flags, paper desk, connect wizard, encryption, options, calendar, backtests, journal, 2FA.
- Round 6: EMIL Trade (terminal clone with Live TV + Live Chat, cockpit ↔ terminal links), named watchlists + share links, API key rotation, Demo Environment.
- Round 7: Instrument Master + symbol normalisation + ■K instrument search. Round 8: charting compare/Bollinger/RSI/levels/layouts.
- Round 9: Morning Brief + AI news impact scoring. Round 10: Portfolio & Exposure + circuit breakers. Round 11: walk-forward + Monte Carlo.
- Round 12: execution protections. Round 13: Ask EMIL grounded in live context. Round 14: instrument research reports. Round 15: Heatmap & Breadth.

Still open (and why)

- SSO between cockpit and EMIL Trade — needs the Raptor Supabase service-role key to mint sessions; today it is a second sign-in.

- Alert delivery (Telegram/email) — envs unset. Company intelligence & screeners, country terminals — need FMP/Finnhub/EIA/FRED keys in Command → Data Providers.
- Payments/wallet/billing — no provider yet (flag crypto_payments OFF). Institutional workspaces/teams — large, not started. Agent-driven live order flow — deliberately not built (autonomous_trading OFF).
- Owner actions: rotate the Gemini/Delta keys pasted in chat, whitelist EMIL's IP on the Delta demo key, set the alert envs, add free data keys.